

Rental Price Estimation Using Regression Models

Negin Azadeh Rezvani
Politecnico di Torino

s338144@studenti.polito.it

Abstract—This project builds a machine learning regression pipeline to predict apartment rental prices based on features like location, size, available amenities, and text descriptions. The main steps in preprocessing include sanity checks, type conversion, and feature engineering. Different regression models were evaluated, and the final pipeline used the best-performing one. The model’s performance was measured using Mean Absolute Error (MAE), which indicated a reliable predictive performance on the evaluation set.

I. PROBLEM OVERVIEW

Predicting apartment rental prices is an important task in real estate. It helps people like renters, owners, investors, and city planners make better decisions [1]. For example, it can help estimate fair prices, plan for affordable housing, or decide if a property is a good investment. However, this is not an easy problem because rent depends on many different factors, and the data can be messy, incomplete, or inconsistent.

In this project, we try to predict the monthly rent of apartments using a machine learning model. We process the data carefully, create new features, and build a regression pipeline. To measure the performance of our model, we use the Mean Absolute Error (MAE), as required, considering it provides a clear measure of average prediction error in the same units as the target variable.

The dataset used in this work consists of 99,487 apartment listings collected from various real estate platforms across the United States. It is divided into two parts:

Development Set: contains 79,589 entries with complete metadata and the target variable price.

Evaluation Set: contains 19,898 entries with the same structure but without the target, used for final prediction.

Each apartment listing includes a variety of features. There are *numerical* features such as square feet, number of bedrooms, and number of bathrooms; *categorical* features like city name, price type, and pets allowed; *geographic* features including latitude and longitude; and *textual* features including the title, body, and listed amenities.

The distribution of rental prices, as it is shown in Fig. 1, is naturally concentrated in a mid-range, with most apartments priced between \$750 and \$1750. While this is expected in the housing market, it creates an imbalance in the data. As a result, the model may learn to predict well for typical prices but struggle with very low or high values that are underrepresented. This can limit the model’s ability to generalize across the full price range, especially for uncommon listings such as luxury apartments or unusually cheap rentals.

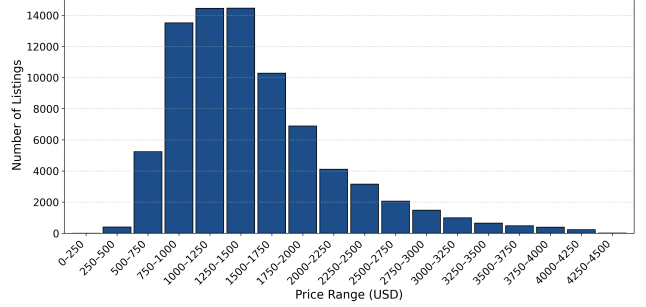


Fig. 1. Distribution of monthly rental prices (capped at 99th percentile).

Some features also require special attention. For instance, the *body* column contains unstructured free text, which cannot be used directly in most machine learning models. To address this, we applied TF-IDF vectorization to transform the text into numerical features, making it suitable for inclusion in the regression pipeline [2]. Features like *cityname* and *source* have many unique values and need special encoding to avoid large and sparse matrices. We also use location data to create geographic clusters and calculate average prices in each cluster.

In this report, we explain our full approach, including preprocessing steps, model selection, and evaluation. Our goal is to build a reliable model that works well even with the challenges of rental data.

II. PROPOSED APPROACH

A. Preprocessing

The dataset was preprocessed by three key steps to ensure data quality, model compatibility, and meaningful feature representation.

1) Sanity Check

An initial sanity check was conducted to evaluate the integrity and structure of the dataset before modeling. This included a systematic inspection for missing values, duplicate records, and outliers in features.

- **Missing Values:** To assess missing data, we calculated the percentage of missing values for each feature. The top 10 features by missingness are reported in TABLE 1. Since the *address* field had 92% missing values, dropping the entire feature was the most reasonable choice. However, for fields such as *pets-allowed* (with 60.7% missing values), we adopted a strategy

grounded in the open-world assumption (OWA). This approach assumes that missing information does not imply falsehood, but rather reflects the possibility of unknown or unrecorded facts. Therefore, we introduced an explicit *Unknown* category to preserve potential signal from these incomplete fields. Less sparse categorical fields, such as *cityname* and *state*, were similarly imputed with placeholder values. Missing entries in *amenities* were filled with empty strings, assuming absence of listing rather than absence of amenities.

TABLE I
TOP 10 FEATURES BY PERCENTAGE OF MISSING VALUES IN THE
DEVELOPMENT SET

Feature	Missing (%)
address	92.01
pets_allowed	60.71
amenities	16.09
state	0.29
cityname	0.29
bedrooms	0.11
bathrooms	0.06
longitude	0.03
latitude	0.03
title	0.00

- **Duplicate Records:** A check for exact duplicates confirmed that the dataset contained no repeated entries, ensuring that each apartment listing is unique.
- **Outlier Detection:** To identify anomalous values, we applied the Interquartile Range (IQR) method to numerical features such as *square-feet*, *bedrooms*, *bathrooms*, and *price*. The proportion of detected outliers was low across all features (e.g., *price*: 4.7%, *square-feet*: 2.8%), hence, no transformation or removal was deemed necessary. These values were retained under the assumption that they may represent legitimate extremes in the rental market rather than data errors.

2) Type Conversion

To ensure compatibility with machine learning models, binary categorical fields were converted into numeric format. The *fee* and *has-photo* columns, originally expressed as *Yes/No*, were converted to *1/0* integers. Such transformation allows these fields to be used directly in the modeling phase without requiring additional encoding.

3) Feature Engineering

To enhance the predictive capacity of the model, a structured feature engineering phase was conducted. Following transformations were applied.

- **City-Level Aggregates:** For each city, the mean and standard deviation of rental prices, along with the total number of listings, were computed. These aggregated statistics were then merged into both the development and evaluation sets, providing localized pricing context for each apartment.

- **Geographic Clustering:** Latitude and longitude coordinates were used to group listings into 200 spatial clusters via KMeans clustering. For each cluster, the mean and standard deviation of rental prices were computed and added as new features, effectively capturing regional pricing patterns that are not explicitly represented in the original location data.
- **Beds-to-Bathrooms Ratio:** A derived ratio feature was introduced to represent the relationship between apartment size and available facilities. To ensure robustness, a safe division operation was applied to avoid division-by-zero errors when computing this ratio.

Finally, all features were categorized based on their data types and modeling requirements. This configuration guided the construction of the preprocessing pipeline, which handled imputation for missing values, standardization of numerical features, encoding of categorical variables, and text vectorization through TF-IDF followed by dimensionality reduction.

This organized transformation strategy resulted in a consistent and information-rich dataset, well-suited for training an effective regression model.

B. Model selection

To predict rental prices effectively, we evaluated a range of regression models that capture different aspects of the data: from linear relationships to complex non-linear patterns. The models tested include: Linear Regression, Random Forest Regression, LightGBM, and CatBoost. Each model brings unique advantages in terms of interpretability, performance, and scalability [3].

• Dummy Regressor

The Dummy Regressor provides a simple baseline by always predicting the mean rental price observed in the training set. Formally, for any input x , the prediction is

$$\hat{y} = \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

where y_i are the true prices in the development set. Although it captures no signal from the features, the Dummy Regressor establishes a lower bound on MAE—any useful model must outperform this naïve predictor.

• Random Forest Regression

Random Forest is an ensemble method based on decision trees. It reduces overfitting by averaging multiple trees trained on bootstrapped subsets of the data and features. The prediction for a sample x is given by:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f_t(x)$$

where T is the number of trees and $f_t(x)$ is the prediction from the t -th tree.

Random Forests are robust to outliers and feature scaling, and they can naturally handle non-linear relationships.

Additionally, they provide useful feature importance metrics, which help in understanding which variables most influence the rental price.

• CatBoost Regressor

CatBoost is another gradient boosting algorithm, specifically optimized for categorical features. It uses ordered boosting to reduce prediction shift and overfitting. Unlike LightGBM, it requires minimal preprocessing of categorical data (e.g., no one-hot or label encoding needed). Its prediction formula is similar to other boosting models:

$$\hat{y} = \sum_{t=1}^T \eta_t f_t(x)$$

where each f_t is a decision tree and η_t is the learning rate.

CatBoost demonstrated competitive performance with minimal hyperparameter tuning. Its automatic handling of categorical features and robustness to overfitting made it an attractive choice in this project [4].

• LightGBM Regressor

LightGBM is a gradient boosting framework that builds trees leaf-wise rather than level-wise, resulting in faster training and improved accuracy. It is well-suited for large datasets with high-dimensional features. At each iteration, LightGBM minimizes the following loss function:

$$L = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

where l is the loss (e.g., MAE), and Ω is a regularization term to control complexity.

LightGBM supports categorical features natively, has built-in support for early stopping, and is highly optimized for both memory and speed. It outperformed Random Forest in our experiments in terms of MAE.

As shown in Fig. 2., MAE is reported for all four models, based on the local evaluation results (Code) and the scores obtained from the leaderboard (Public).

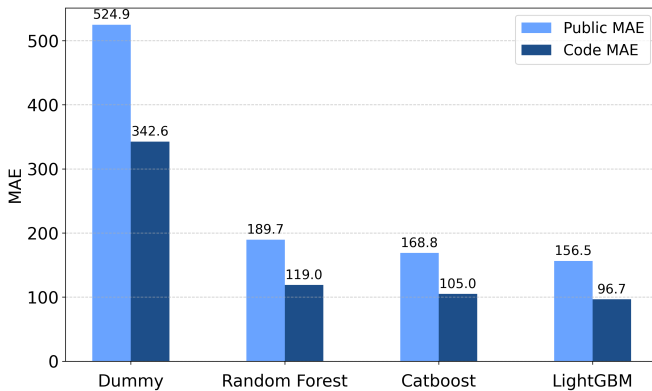


Fig. 2. Comparison of MAE by model (public vs. code).

LightGBM showed the best performance with a Code MAE of 96.7, followed by CatBoost (105.0) and Random Forest (119.0). In contrast, the Dummy model produced a much higher MAE of over 340, that confirms that learning-based models can effectively use the apartment features to make more accurate predictions. Based on these results, we selected LightGBM as the final model for this project due to its superior performance.

C. Hyperparameter Tuning

Due to the large size of the dataset and the complexity of the LightGBM model, running a full grid search over all combinations of hyperparameters was not practical with our available computing resources. The initial plan was to first apply a *Randomized Search* to identify promising parameter ranges, and then perform a more focused *Grid Search* around the best results. However, due to time and resource limitations, we decided to rely only on the randomized approach.

We used `tRandomizedSearchCV` with 20 randomly selected parameter combinations, each evaluated using 5-fold cross-validation. To ensure that all price levels were represented in each fold, we stratified the data based on price deciles. Additionally, we applied a log-transform to the target variable ($y' = \log(1 + y)$) to reduce the influence of extreme price values. The parameter ranges were selected based on LightGBM best practices and our earlier experiments.

TABLE II
RANDOMIZED SEARCH PARAMETER DISTRIBUTIONS

Hyperparameter	Candidate Values
learning_rate	{0.01, 0.03, 0.05}
num_leaves	{128, 256, 512}
subsample	{0.6, 0.8, 1.0}
colsample_bytree	{0.6, 0.8, 1.0}

The selected ranges reflect a balance between model flexibility and overfitting control:

- *learning_rate*: lower values improve stability at the cost of more trees;
- *num_leaves*: larger leaf counts capture more non-linear interactions;
- *subsample* and *colsample_bytree*: control row and feature bagging to reduce overfitting.

The randomized search was configured as follows:

- `n_iter` = 20 random draws
- `scoring` = `neg_mean_absolute_error`
- 5-fold StratifiedKfold on price deciles
- `n_jobs` = -1 for parallel execution

After fitting, the best hyperparameter combination was:

TABLE III
BEST HYPERPARAMETER COMBINATION

Hyperparameter	Selected Value
learning_rate	0.03
num_leaves	256
subsample	0.8
colsample_bytree	0.6

III. RESULTS

The tuned LightGBM regression pipeline demonstrated strong predictive performance under 5-fold cross-validation, stratified by deciles of the log-transformed target variable. The fold-wise Mean Absolute Errors (MAE) were:

[0.08885, 0.08974, 0.08874, 0.08969, 0.08789]

with an average MAE of **0.08898** on the log-transformed scale. After applying the inverse transformation using `expm1()`, the model achieved a MAE of 141.525 on the public leaderboard. This close match between internal validation and external evaluation confirms the reliability and generalization capability of the pipeline.

To interpret the model's predictions, we examined the split importance scores provided by LightGBM's built-in `feature_importances_` attribute. This metric reflects the number of times each feature was used to split decision nodes across all trees in the ensemble. Fig.3. illustrates the top 15 most influential features.

The results indicate that the model relies heavily on a combination of structured and engineered variables.

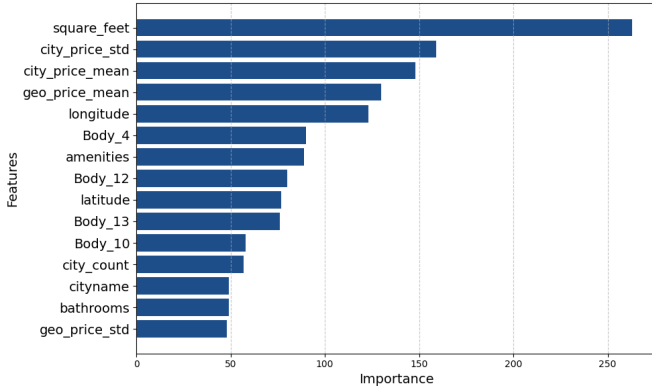


Fig. 3. Top 15 most influential features

IV. DISCUSSION

The results confirm the strength of gradient boosting methods in modeling complex real estate pricing dynamics. LightGBM consistently outperformed the other evaluated models, achieving the lowest Mean Absolute Error (MAE) on both our internal cross-validation (88.980) and the public leaderboard (141.525). Its ability to handle heterogeneous feature types, capture non-linear relationships, and scale to large datasets

made it particularly suited for this task [5]. In contrast, simpler models such as the Dummy regressor yielded high error, and even robust learners like Random Forest and CatBoost did not match LightGBM's accuracy.

Feature importance analysis (Fig. 3.) further supports this outcome. The most predictive feature was *square_feet*, followed by city and geographic statistics. These results suggest that both the physical attributes of a property and its broader geographic context are key to determining rental prices.

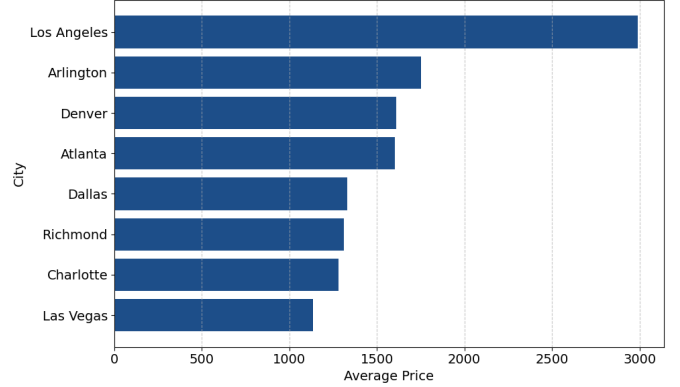


Fig. 4. Top 8 Most Expensive Cities

City-level effects are also visible in the average price plot (Fig. 4.), where high-cost cities such as Los Angeles and San Diego dominate the upper end of the distribution. Incorporating city-specific price aggregates thus enabled the model to adjust predictions based on local market expectations, improving generalization.

Several important features come from the *body* text field, transformed using TF-IDF. Although these features are not directly interpretable, their high importance suggests that descriptions (like renovations or views) influence price. Future work could use language models (e.g., BERT) to better understand how text content affects predictions.

Overall, this study illustrates the effectiveness of LightGBM when paired with careful preprocessing and domain-specific feature engineering. It also emphasizes the importance of balancing accuracy with interpretability, especially in applied pricing tasks where transparency is valued alongside precision.

REFERENCES

- [1] H. R. Chan, "Rent price prediction with advanced machine learning methods: A comparison of california and texas," *Highlights in Science, Engineering and Technology CSIC*, vol. 85, pp. 501–507, 2024.
- [2] S. Bushuyev, D. Bushuyev, D. Kravtsov, *et al.*, "Machine learning model for house price predicting based on natural language text data analysis," in *MoMLet-2024*, 2024.
- [3] R. Shekh, A. Neyaz, A. Ahmed, and A. Singh, "Machine learning for rental price prediction: Regression techniques and random forest model," *ResearchGate*, 2024.
- [4] R. Guo *et al.*, "Research on renting price prediction based on machine learning," *EAI Endorsed Transactions on Scalable Information Systems*, 2023.
- [5] P. Adzanoukpe, "Predicting house rental prices in ghana using machine learning," *arXiv preprint arXiv:2501.06241*, 2025.